working title

A robot's code needs to sense the environment, control the hardware, and communicate with other robots. Current programming languages do not provide the necessary hardware platform-independent abstractions, and therefore, developing robot applications require detailed knowledge of signal processing, control, path planning, network protocols, and various platform-specific details. Further, porting applications across hardware platforms becomes tedious.

We present *Koord*—a domain specific language for distributed robotics—which abstracts platform-specific functions for sensing, communication, and low-level control. *Koord* makes the platform-independent control and coordination code portable and modularly verifiable. It raises the level of abstraction in programming by providing distributed shared memory for coordination and port interfaces for sensing and control. We have developed the formal executable semantics of *Koord* in the $\mathbb{K}$ framework. With this symbolic execution engine, we can identify assumptions (proof obligations) needed for gaining high assurance from *Koord* applications. We illustrate the power of *Koord* by creating and verifying three applications: formation flight, distributed task allocation, and distributed mapping. We show how platform-independent proof obligations can be discharged using *Koord*'s Z3 backend while platform-specific proof obligations can be checked (a) by simulating the applications on the high-fidelity *Koord* simulator, (b) by testing the applications on actual vehicle and quadcopter platforms, and (c) by verifying the obligations using physics-based models and hybrid verification tools.

Are we doing all 3? What is the key takeways from all this? is it effective? What did we learn?

1 INTRODUCTION

Motivation and goals. Distributed robotic applications could transform manufacturing [Gauthier et al. 1987; Pires and Da Costa 2000], transportation [Gerla et al. 2014; Guo and Yue 2012], agriculture [Blender et al. 2016; R Shamshiri et al. 2018], delivery [Mosterman et al. 2014], and mapping [Thrun et al. 2002]. Following the trends in cloud, mobile, and machine learning applications, programmability is key in unlocking this potential as robotics platforms become more open, and hardware developers shift to the applications marketplace. Available domain specific languages (DSL) for robotics are tightly coupled with platforms, and they combine low-level sensing, communication, and control tasks with the application-level logic (see Section 7 for more details). This tight-coupling and the attendant lack of abstraction hinders application development on all fronts—portability, code reuse, and verification and validation (V&V).

Reliably building an application for a collection of coordinating robots involves two very different types of thinking: (1) Correctness arguments for coordination algorithms under concurrency and asynchrony are *hardware platform independent*, and use techniques from formal verification and distributed computing. (2) Correctness arguments for physical interactions of the robots (e.g., sensing and motion control) under noise and disturbances are *platform dependent*, and use techniques from control theory. Sentence about closest existing works that do not do the job and citations. E.g., the whole hybrid systems literature is too abstract from the programming point of view. In this paper, we enable reliable engineering of distributed robotics applications by separating the platform independent and platform dependent concerns, and creating a programming language and a tool chain that effectively support both (1) and (2). Rewrote the above para

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

, ,

© Association for Computing Machinery.

What is this para accomplishing? Is it saying why this is hard? Its a bit weak. Is it giving a concrete example? That may be useful. Is the ROS inspiration important? What does that buy us in terms of advancing the contributions. To further this idea of separating the concerns, we draw insights from one of the popular programming frameworks that supports various hardwares, Robot Operating System (ROS) [Quigley et al. 2009]. The framework roughly separates reusable *platform-independent* software modules from those specific to each platform. For instance, the module for sharing sensor data is platform-independent. Even though each platform may use different kinds of sensors, it is assumed the data will be converted to platform-independent formats beforehand. Our intuition is to generalize the platform-independent formats to *platform-independent abstractions* over physical behaviors of different platforms. For distributed package delivery with mobile robots as an example, we can define abstract interfaces for assigning the destination and checking if the destination is reached, and abstract away the implementation of way-point tracking and navigation on each robot. Application code using such interfaces will be portable, with appropriate platform-specific implementations of these abstractions, e.g., steering controller for car, thrust controller for a quadcopter, and GPS or indoor localization subroutines. In addition, if the abstractions are formally specified, we would like the correctness proof of the whole system be decomposed into (1) *platform-independent proof obligations* for distributed programs with respect to the abstraction for all target platforms, and (2) *platform-dependent proof obligations* showing that each robot platform in use implements the abstraction. This decomposition will directly enable our idea of plugging-in different analyses from different communities. However, it is unclear such decomposition can exist for arbitrary combinations of distributed programming models and platform models. And to make things more complicated, a detailed model of the platform (e.g., dynamic models for cars, wheel friction, engine torque, etc.) is often not available in practice.

In this paper, we present *Koord*: a language, its formal executable semantics, and supporting verification and testing tools for programming distributed robotic systems. A user can code distributed robotics applications using the *Koord* language. This *Koord* program can be deployed to cars and quadcopters, simulated with virtual vehicles, and verified via our decomposition approach and various existing verification tools. Our technical contributions are the following:

First, *Koord* provides abstractions and language constructs for coordination and control that separate the *platform-independent* program logic, such as distributed decision making, from *platform-dependent* control tasks for sensing, planning, and actuation. This makes *Koord* applications very succinct and hence readable. A *Koord* program for a set of robots to form a line is only 10 lines (see Figure 3). In another application, robots coordinate and visit waypoints in a mutually-exclusive fashion, while avoiding collisions—all in 50 lines of *Koord* code. A third application, discussed briefly here, accomplishes distributed mapping. Development of these and other nontrivial applications, demonstrate the utility of the *Koord* abstractions.

Secondly, we have developed the executable semantics of *Koord* in $\mathbb{K}$ [Rosu and Serbanuta 2014]. To our knowledge this is the first formalization of a programming language for distributed cyber-physical systems which has also been deployed on actual platforms. We show that this executable semantics enables us to plug-in different verification techniques for the platform-independent and the platform-dependent proof obligations for *Koord* applications. We are able to decompose and verify geofencing and collision-avoidance invariants for the abovementioned *Koord* applications. We show that (1) platform-independent proof obligations can be formulated as inductive invariance checks and can be effectively discharged use the right words here Z3/Koord prover/What are all the tables etc. saying about effectiveness? (2) Platform-dependent proof obligations can be formulated as reachability queries for hybrid dynamical systems and can be effectively discharged using a number of standard tools including the simulation-driven reachability tool DryVR [Fan et al. 2017] we use in this paper.

Old text Executable semantics enabled us to plug-in different platform models or even black-box executables. A major benefit arising from the semantics is that

Koord programs can be deployed to different platforms with a *Koord* execution engine once the implementation of abstractions is plugged-in, and the program execution is guaranteed to conform to the formal semantics. A reference *verified* interpreter for *Koord* is auto-generated thanks to $\mathbb{K}$ framework. A multi-platform *Koord* execution engine deployed to actual platforms is also available in [Author 2019].

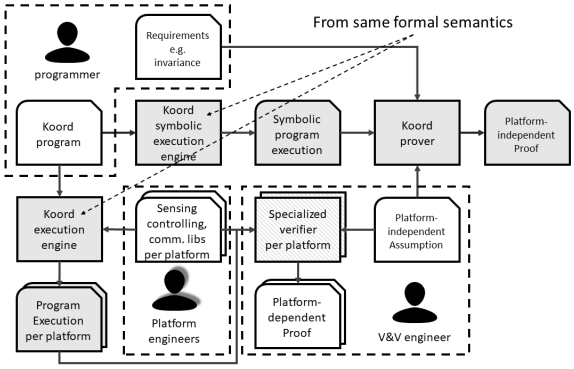

Fig. 1. *Koord* enables programming distributed robotics applications using high-level abstractions, and analyzing those applications using a combination of symbolic executions, simulations, and simulation-driven verification.

I guess we will drop/simplify this figure.

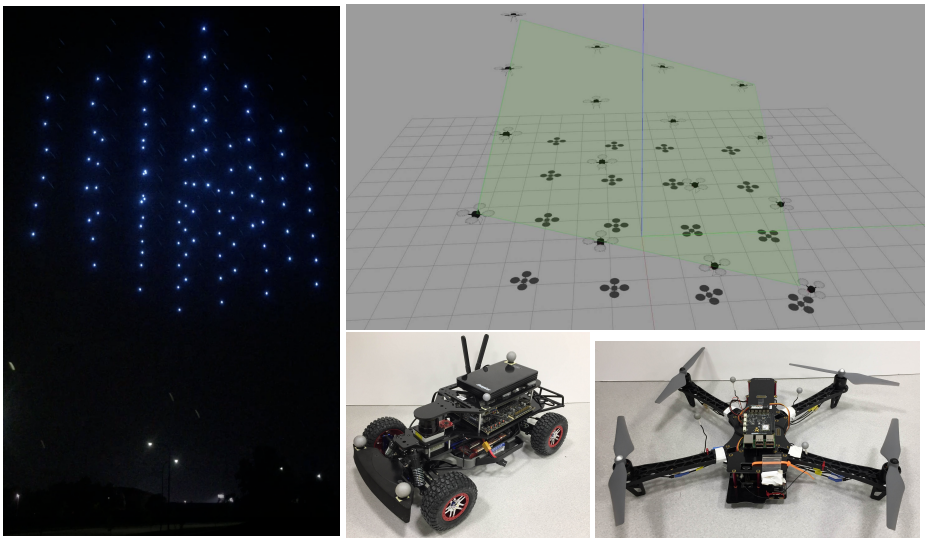

Fig. 2. Swarm formation show by FireFly Inc. (*Left*). Simulation of shape formation (*Top Right*). Cars and quadcopters we can deploy and test *Koord* programs.

2 OVERVIEW

We will discuss the key features of the *Koord* programming system with an example application for formation control. This application makes a collection of quadrotors form a pattern as is seen in drone shows (Figure 2). The *Koord* application LineForm of Figure 3 is a basic version that makes a collection of robots line up uniformly between two extremal robots. Small modifications to the code can make the drones form other shapes like squares, cubes, and stars.

2.1 The *Koord* language

Koord is an event-driven language in which application programs use *shared variables* for coordination across robots and *ports* for interaction with hardware-specific control subroutines.

```
1   using Motion:
2     sensors: pos psn
3     actuators: pos target
4
5   allread: pos x
6   TargetUpdate:
7     pre True
8     eff if not(pid == N_sys − 1 or pid == 0):
9       Motion.target = mid( [x[pid+1],x[pid−1]])
10      x[pid] = Motion.psn
```

Fig. 3. *Koord* program LineForm for a set of robots to form a line.

Port abstractions for platform-dependent interactions. A *Koord* program interacts with the sensors and controllers of the robot platform through *sensor* and *actuator* ports. These ports can be seen as special variables: the program can read data from the sensor ports and can write data to actuator ports. For example, LineForm uses a *module* (library) called *Motion* which provides a sensor port called *position* that publishes the robot's position, and an actuator port called *target* for specifying a target position. Thus, these ports provide an abstraction over various possible sensor and controller implementations and environments. Implementations of the modules will usually be hardware specific. For example, we used an implementation of the *Motion* module for a quadcopter uses an indoor camera based positioning system to update the *position* port, and it uses an RRT based [LaValle 1998] path planner and motion controller. On the other hand, for a small racing vehicle platform, we used another implementation of *Motion* with the same indoor positioning system but a different model-predictive controller.

Local and shared variables for platform-independent agent interactions. Koord programs can have *local* variables similar to most programming languages. In addition, they can also use *shared* variables for participating robots to communicate with each other. At Line 5 in Figure 3, an **allread** variable, x, is a shared array which all robots can read from, but each robot `pid` can only write to x[`pid`]. This shared array is used to share the current position of each robot with all other robots. LineForm uses (a) the unique integer identifier `pid` for itself and (b) the number N_{sys} of all participating robots. A detailed list of system parameters will be discussed later in Section 3. For multiple robot programs writing to shared variables *Koord* provides concurrency control with mutual exclusion and **atomic** blocks, (see Task in Section 6).

2.2 Semantics and invariant properties

(1) more focus on executable semantics, why its better than other formal semantics, and why the veri-

(1)

We have developed the full semantics of *Koord* using $\mathbb{K}$ [Rosu and Serbanuta 2014], and present the details in Section 3.

In a distributed robotics setting, given a system with N_{sys} robotics, instances of the same *Koord* program are executed by each participating robot to solve problems collectively. The execution of the *Koord* program advances in a synchronous, *round-by-round* fashion, where each round lasts for δ time, and $\delta > 0$ is a system-specific execution parameter. During this period, the robots compute, move, and communicate with each other through distributed shared memory.

The execution semantics of any applications for multi-robot systems are complicated by issues of asynchrony, consistency of shared memory, and interactions between software and the physical environment. The $\mathbb{K}$ rewriting engine makes the formal language semantics *executable*, and enables exhaustive exploration of non-deterministic behaviors of *Koord* applications. The major advantage of using executable formal semantics for formal analysis and verification is that the gap between a theoretical model and programming language implementation is reduced. (2) In Section 4, we present a method for checking invariant properties using symbolic execution of the *Koord* semantics in $\mathbb{K}$.

(2) we may be asked questions about this, how much detail do we go into about this

For LineForm, a natural requirement is to restrict all robots to stay within a certain safe area, at all times (Geofencing). More precisely, given a (hyper)rectangle $rect(\mathrm{a}, \mathrm{b})$ defined by its two corners a and b, if all robots are initialized within the rectangle, then all robots should always stay in the rectangle, as should their shared positions.

This requirement can be stated as:

Invariant 1. $\bigwedge_{i \in \mathtt{ID}} \left(M.pos_i \in rect(\mathrm{a}, \mathrm{b}) \wedge x_i \in rect(\mathrm{a}, \mathrm{b}) \right)$

where $M.pos$ is the shorthand for $Motion.position$.

Using *Koord*'s supporting proof tools, an invariant like the above can be established in two steps: first, assuming that all the robot positions are in $rect(\mathrm{a}, \mathrm{b})$, we show that the targets computed by LineForm are also in $rect(\mathrm{a}, \mathrm{b})$.

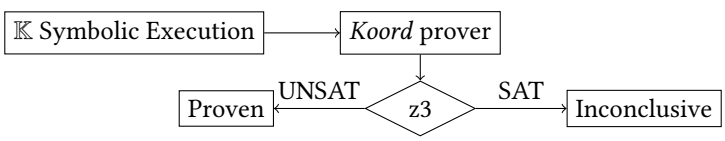

Fig. 4. $\mathbb{K}$ semantics based invariant checking for *Koord*.

The $\mathbb{K}$ semantics of *Koord* allows us to construct the symbolic post states of the *TargetUpdate* event and prove Invariant 1 using the *Koord* prover (Figure 4) as detailed in Section 5.

The second step is to show that for any robot, assuming that its current position and computed targets are in $rect(\mathrm{a}, \mathrm{b})$, the controller implementing *Motion* module indeed keeps the robot inside $rect(\mathrm{a}, \mathrm{b})$.

For this step, one has to reason about how each robot moves when its implementation of the *Motion* module is given a *target*. *Koord* helps identify and decompose the overall proof into assumptions that the *Module* implementations need to guarantee. For example, we can state an assumption that may be used for Invariant 1 as:

Assumption 1.

$$\forall t \in [0, \delta], f(M.pos, M.tgt, t) \subseteq rect(M.pos, M.tgt),$$

where $M.tgt$ is the shorthand for $Motion.target$, f is a function specifying the position of the robot at time t, moving to $M.tgt$, from $M.pos$. This assumption states that the robot's *Motion* module should ensure that it is moving within the bounding rectangle between its position and target within the duration of a round. In general, if assumptions for correctness are too strong under specific

scenarios, we can either revise the assumptions or modify the invariant so that weaker assumptions may be sufficient. Using simulation based verification techniques, we discover that Assumption 1 may be violated, because the inertia of the robot may force it go out of bound temporarily. We see through an example in Section 5 how this scenario can be detected.

2.3 Implementation

While the formal semantics is executable and allows us to generate execution traces of the system in terms of *configurations* (specified later in Section 3.3, it is not deployed on actual robots. It is also generally inconvenient to use the executable semantics to generate simulation traces specifically for robot-dynamics to use in simulation based verification techniques used to validate port assumptions mentioned above.

For those purposes, we use an implementation of *Koord* as presented in [Author 2019] for simulation and deployment. This implementation includes a compiler (written using Antlr4 in Java) for *Koord* as well. The compiler generates Python application code from abstract syntax trees, which in turn are generated from from the BNF grammar of the *Koord* language presented in Section 3.3.

This implementation supports the execution of *Koord* programs through Python middleware, whcih implements shared memory through message passing. It includes a lock-based mutual exclusion mechanism for shared variables and uses message passing to synchronize the rounds of execution. The details of this toolchain are not the focus of this paper, we instead focus on the programming language design, semantics, and the formal analysis capabilities enabled by the same.

3 THE *KOORD* LANGUAGE

In this section, we present the syntax and the semantics of *Koord*. When a *Koord* application is deployed on a fleet of N_{sys} robots, each robot runs an instance of the same program. There is a known set of identifiers $\texttt{ID} = \{0, 1, \ldots, N_{sys} - 1\}$, and each robot is assigned a unique index $\texttt{pid} \in \texttt{ID}$.

3.1 Syntax

Koord abstractions simplify distributed coordination and interactions with the physical environment. Figure 5 shows the core grammar of *Koord* syntax. Each robot program consists of (a) declarations of the interfaces between the program and the sensor/actuator modules, (b) declarations of shared and local program variables, and (c) events, consisting of preconditions and effects. Robot programs (rule *Program*) first can import sensor/actuator modules. The module import grammar production specifies the interfaces or ports: it contains all input and output ports for actuators (*APorts*) and sensors (*SPorts*) that the program uses. To summarize, there are following three types of names for reading/writing values:

(i) *Sensor and actuator ports* are used to read from sensor ports and write to actuator ports of controllers.
(ii) *Local program variables* record the state of the program.
(iii) *Distributed shared variables* are used for coordination across robots. All shared variables can be read by all participating robots; an **allwrite** variables can be written by any participating robot; while an **allread** variable can be written only by a single-writer.

Users can then optionally specify a statement to set the initial values of program variables (rule *Init*). The main body of the program is a sequence of events (rule *Event*) which include a Boolean **pre**condition and an **eff**ect. The effect of an event is also a statement (rule *Effect*).

A statement (rule *Stmt*) in *Koord* resembles those in most imperative languages and includes conditional statements, function calls, assignments, blocks of statements, atomic statements for mutual exclusion, etc. Mutual exclusion is always an essential feature when shared variables are

```
Program   ::= Defs Module+? DeclBlock Init? Event+

Defs      ::= [FuncDef+]? [AdtDef+]?
FuncDef   ::= def fun identifier(Param+) : Stmt+
AdtDef    ::= def adt identifier : Decl+
Param     ::= Type identifier

Module    ::= using module identifier : SPorts APorts
SPorts    ::= sensors : Decl+
APorts    ::= actuators : Decl+

Decl      ::= Type identifier | Type identifier =Val
Type      ::= int | float | bool | pos | adt
            | Type[Int] | List ⟨Type⟩ | Queue ⟨Type⟩

DeclBlock ::= AWDecls ARDecls LocalDecls
AWDecls   ::= allwrite : Decl+
ARDecls   ::= allread : Decl+
LocalDecls ::= local : Decl+

Init      ::= init : Stmt+
Event     ::= identifier : pre (Cond) eff : Stmt+

Expr      ::= AExpr|BExpr
AExpr     ::= AExpr AOp AExpr
            | Expr++ | -AExpr | Var | AVal
AOp       ::= + | − | ∗ | /
BExpr     ::= Expr RelOp Expr|Expr COp Expr
            | notExpr | Var | BVal
RelOp     ::= ≥|≤|≥|==|>|<|≠
COp       ::= and | or

Stmt      ::= Assign | FnCall | Atomic
            | Ite | Loop | Return
Assign    ::= Var = Expr
Ite       ::= if BExpr Stmt+ [else Stmt+]?
FnCall    ::= identifier(Expr+)
Atomic    ::= atomic : Stmt+
Loop      ::= for identifier in AExpr : Stmt+
Return    ::= return Expr | return

Var       ::= identifier | identifier[Expr]
            | identifier.identifier
Val       ::= AVal | BVal
AVal      ::= Int | Float
BVal      ::= Bool
```

Fig. 5. Core *Koord* program syntax.

involved. *Koord* provides a locking mechanism using the keyword **atomic** to update the shared variable safely. The user can also define functions and abstract data types (tuples of the basic data types). These features enable a natural separation of the discrete computational (platform independent) and dynamic (platform dependent) behaviors. To discuss these behaviors, we need to establish the notion of system and robot state.

3.2 Configurations

Recall from Section 2 the semantics of a *Koord* program execution is based on synchronous rounds. Each round is divided into *program transitions* and *environment transitions* that update the *system configuration*. In each round, each robot performs at most one event. The update performed by a single robot executing an event is modeled as an instantaneous transition that updates the program variables; however, different events executed by different robots may interleave in an arbitrary order. In between the events of successive rounds, $\delta > 0$ duration of time elapses, the program variables remain constant while the values held by the sensor and actuator ports may change. These are modeled as environment transitions that advance time as well as the sensor and actuator ports. Thus, each round consists of a burst of (at most N_{sys}) event transitions followed by an environment transition. This is a standard model for synchronous distributed systems where the speed of computation is much faster than the speed of communication and physical movement [Attiya and Welch 2004; Lynch 1996].

We now describe the system state, or *system configurations* which we use to formalize *Koord* semantics.

System configurations. A *system configuration* is a tuple $\boldsymbol{c} = (\{L_i\}_{i \in \mathtt{ID}}, S, \tau, \mathit{turn})$, where

(i) $\{L_i\}_{i \in \mathtt{ID}}$ or $\{L_i\}$ in short is an indexed set of *robot configurations*–one for each participating robot. L_i refers to the configuration of the i-th element, i.e., the i-th robot in the system.
(ii) $S : \mathit{Var} \mapsto \mathit{Val}$ is the *global context*, mapping all shared variable names to their values.
(iii) $\tau \in \mathbb{R}_{\geq 0}$ is the *global time*.

$$\frac{\forall i \in \texttt{ID}, L_i.turn = \texttt{env}}{(\{L_i\}, S, \tau, \texttt{prog}) \rightarrow_G (\{L_i\}, S, \tau, \texttt{env})} \ \textsc{EndProgTrans}$$

$$\frac{\begin{array}{c}\forall i \in \texttt{ID}, \langle S, L_i\rangle \rightarrow_{env} \langle S, L_i'\rangle \\ \wedge\, L_i.turn = \texttt{env} \wedge L_i'.turn = \texttt{prog}\end{array}}{(\{L_i\}, S, \tau, \texttt{env}) \rightarrow_G (\{L_i'\}, S, \tau + \delta, \texttt{prog})} \ \textsc{EnvTrans}$$

$$\frac{\begin{array}{c}\exists i \in \texttt{ID}, \langle S, L_i, \oplus\rangle \rightarrow_{stmt} \langle S', L_i', \cdot\rangle \\ \wedge\, L_i.turn = \texttt{prog} \wedge L_i'.turn = \texttt{env}\end{array}}{(\{L_i\}, S, \tau, \texttt{prog}) \rightarrow_G (\{L_i'\}, S', \tau, \texttt{prog})} \ \textsc{EventTrans}$$

Fig. 6. System semantic rules for *Koord*.

(iv) $turn \in \{\texttt{prog}, \texttt{env}\}$ is a binary *bookkeeping* variable determining whether program or environment transitions are being processed.

Bookkeeping variables are invisible in the language syntax, and only used in the semantics. Note that, while *turn* for the system configuration may be a bookkeeping variable, it is directly used to achieve the separation of platform dependent and platform independent concerns in the semantics. We now define the robot configurations which define the state of every robot in the system.

Robot configurations. A *robot configuration* is used to specify the semantics of each robot. As a *Koord* program is run on a system of robots, each participating robot would have its own set of module ports and local variables, along with a local copy of each shared variable. Given a *Koord* program P, we can define *Var* be the set of variables, *Val* be the set of values that an expression in *Koord* can evaluate to, *CPorts* be the set of sensor and actuator ports of the controller being used, and *Events* the set of events in P. A robot configuration is a tuple $L = (M, cp, turn)$, where

(i) $M : Var \mapsto Val$ is its *local context* mapping both local and shared variables to values. Note that this implies M includes a copy of shared variable values.
(ii) $cp : CPorts \mapsto Val$ is the mapping of sensor and actuator ports to values.
(iii) $turn \in \{\texttt{prog}, \texttt{env}\}$ is a bookkeeping variable indicating whether this robot is executing a program or environment transition.

For readability, we use the dot (".") notation to access components of a robot configuration L. For example, $L.M$ means accessing the local context M in the tuple L.

3.3 Semantics

The execution semantics for a distributed *Koord* program captures the separation of the discrete and the dynamic behaviors (the program and environment transitions) of the robots through *rewrite rules*. Rewrite rules at various levels : *System*, *Robot* and *Expression* are required to specify the semantics of a *Koord* program, and allow us to create a framework for formal analysis.

System semantics. For system-level semantics, the rewrite rule is a mapping from an initial system configuration to a set of configurations. It has the type

$$\rightarrow_G \subseteq \mathbb{C} \mapsto \wp(\mathbb{C}),$$

where $\mathbb{C}$ is the set of all possible system configurations.

The bookkeeping variable *turn* is used by the system to determine whether the system (all robots in the system) is performing a program transition, or an environment transition. The system executes an environment transition only when the local *turn* of each robot is env. After all robots enter the env turn, rule EndProgTrans sets the global *turn* from prog to env indicating the end of program transition, and an environment transition will occur afterwards.

$$\frac{\begin{array}{l} L.turn = \texttt{prog} \\ \wedge \text{“}Name\text{: } \textbf{pre:} \ Cond \ \textbf{eff:} \ Body\text{”} \in Events \wedge Cond(S, L) \end{array}}{\langle S, L, \oplus \rangle \rightarrow_{stmt} \langle S, L, Body \rangle} \ \textsc{SelectEvent}$$

$$\langle S, L, \oplus \rangle \rightarrow_{stmt} \langle S, L, \cdot \rangle \ \textsc{SkipEvent} \qquad \langle S, (M, cp, \texttt{prog}), \cdot \rangle \rightarrow_{stmt} \langle S, (M, cp, \texttt{env}), \cdot \rangle \ \textsc{EndEvent}$$

$$\frac{\forall x \in Keys(S), M' = M[x \mapsto S[x]] \wedge cp' = f(cp, \delta)}{\langle S, (M, cp, \texttt{env}) \rangle \rightarrow_{env} \langle S, (M', cp', \texttt{prog}) \rangle} \ \textsc{RobotEnv}$$

Fig. 7. Partial per robot semantic rules for *Koord*.

Rule EnvTrans shows the evolution of the system configuration after the rule EndProgTrans is applied.This rule synchronizes the *environment* transitions of the robots and advances the global time from τ to $\tau + \delta$.

During a program transition, each robot executes a sequence of statements, or rewrite rules for statement semantics of type

$$\rightarrow_{stmt} \subseteq (\mathbb{S} \times \mathbb{L} \times (Stmt \cup \{\oplus, \cdot\})) \mapsto \wp(\mathbb{S} \times \mathbb{L} \times Stmt \cup \{\cdot\}),$$

where *Stmt* refers to the set of all possible statements allowed by *Koord* syntax. We use internal syntactic structures '$\oplus$' and '$\cdot$', which are are not in *Koord* themselves, but are used to represent control flow in *Koord* programs in the semantics, as we will see in the discussion on per-robot semantics. '$\oplus$' is to denote nondeterministic selection of events, and '$\cdot$' is to indicate an "empty" statement.

This relation takes as input a tuple of (1) a global context, (2) a robot configuration, and (3) a statement, and maps it to a set of such tuples. Rule EventTrans expresses that starting from a system configuration $\boldsymbol{c} = (\{L_i\}, S, \tau, \texttt{prog})$, a robot i with the configuration L_i starts by selecting an enabled event, executes the event via a sequence of $\rightarrow_{stmt}$ rewrites, and sets its own *turn* to env at the end of the event execution. The system goes from a configuration $\boldsymbol{c}$ to $\boldsymbol{c}' = (\{L_i'\}, S', \tau, \texttt{prog})$, with possibly different robot configurations and global context depending on whether any statement executed resulted in writes to shared variables. The system can display nondeterministic behaviors arising from different robots executing their events in different orders.

We now go into some detail to discuss the $\rightarrow_{stmt}$ rewrites which specify the behavior of each robot during a program transition. These rules are used to individual *robot* configurations.

Per robot semantics. Events are the main computational blocks in a *Koord* program. We present the core semantic rules for event execution by a robot executing a Koord program. Rule SelectEvent in Figure 7 shows that any event may be executed when the precondition *Cond* is evaluated to true, and by replacing $\oplus$ with the event effect *Body*, it ensures only one event is selected and executed. The event effect is then executed following the semantics of each statement in *Body*. Rule SkipEvent allows the robot to skip the event completely. At the end of the event, the sequence of statements becomes empty '$\cdot$'. Rule EndEvent then makes sure the *turn* of the robot is set to env indicating that an environment transition will occur afterwards.

While $\rightarrow_{stmt}$ rewrites define each robot's behavior during a program transition, we separate the platform-dependent semantics of how each robot interacts with environment (including other robots) using environment transition rules of the type

$$\rightarrow_{env} \subseteq (\mathbb{S} \times \mathbb{L}) \mapsto \wp(\mathbb{S} \times \mathbb{L}),$$

which takes a global context and a robot configuration as input. Rule RobotEnv simply states that the new local context M' is the old local context M updated with the global context S; thus ensuring that all robots have consistent shared variable values before the next program transition.

$$\frac{\langle S, L, St\rangle \rightarrow_{stmt} \langle S', L', St'\rangle}{\langle S, L, St\ StList\rangle \rightarrow_{stmt} \langle S', L', St'\ StList\rangle}\ \textsc{StmtSeq1}$$

$$\langle S, L, \cdot\ StList\rangle \rightarrow_{stmt} \langle S, L, StList\rangle\ \textsc{StmtSeq2}$$

$$\frac{x \in Keys(S) \wedge x \in Keys(L.M) \wedge L'.M = L.M[x \mapsto v]}{\langle S, L, x = v\rangle \rightarrow_S \langle S[x \mapsto v], L', \cdot\rangle}\ \textsc{SvarAssign}$$

$$\frac{x \notin Keys(S) \wedge x \in Keys(L.M) \wedge L'.M = L.M[x \mapsto v]}{\langle S, L, x = v\rangle \rightarrow_S \langle S, L', \cdot\rangle}\ \textsc{LvarAssign}$$

Fig. 8. Example statement level semantic rules for *Koord*.

(3) need to make f more illustrative of dynamics, move this paragraph to robot semantics?

(3) To define the executable $\mathbb{K}$ semantics of *Koord* applications, we have to provide executable descriptions for the environment transitions. The type of this executable object (f) is defined by *CPorts*, namely, $f : [CPorts \mapsto Val] \times \mathbb{R}_{\geq 0} \mapsto [CPorts \mapsto Val]$. That is, given old sensor and actuator values and a time point, f should return the new values for all sensor and actual ports.

New sensor readings cp' are then obtained by evaluating the black-box dynamics f with time δ. In an actual execution, the controller would run the program on hardware, whose sensor ports evolve for δ time between program transitions. This formalization allows the ports to behave arbitrarily over δ-transitions. Hence in verification, additional assumptions over the behavior of the sensor and actuator ports are needed. Finally, the *turn* of the robot is set back to `prog`.

Aside from such rules,during program transitions, *Koord* semantics include rewrite rules showing the impact of the shared memory abstractions on the configurations of each of the robot, control flow, etc. We illustrate a few of these rules, in Figure 8 with rules such as Rule StmtSeq1 and StmtSeq2 showing how a statement representing a sequence of statements is executed.[1] Rule LvarAssign and Rule SvarAssign show the semantic rules for local and shared variable assignment respectively are also examples of statement level rules. Evaluating these rules requires in turn, expression level rules, which include variable lookup, arithmetic, boolean , and relational operations amongst others. We present a few illustrative examples below.

Expression-level Semantics. The expression level semantics are given by rewrite rules of the type

$$\rightarrow_E \subseteq (\mathbb{S} \times \mathbb{L} \times \mathbb{E}) \times (\mathbb{S} \times \mathbb{L} \times \mathbb{E}),$$

where $\mathbb{S}$ is the set of all possible global contexts S, $\mathbb{L}$ refers to the set of all possible values for configurations of an robot, and $\mathbb{E}$ refers to the set of all possible expressions allowed by the language syntax.

The variable lookup rule Var-Lookup-rule states that every robot has a local copy of every variable in the program, and if an robot is evaluating an expression involving variable x, it will replace x with the current value v from the local context M. $M[x]$ here obtains the value corresponding to the key x.

We also present the rules for addition (Aexpr-Add-rule). They are fairly standard: the execution first evaluates the left subexpression, then the right subexpression given that left is already evaluated to a value and finally adding the two values. We omit the similar rules for other arithmetic, boolean, and relational operations.

The semantic rules we discussed also realize the distributed nature of the design of the *Koord* system. The memory consistency model, and the synchronization model of *Koord* have been designed to enable separation and analysis of the program and environment transitions as well.

[1]The full core semantics will be made available with the supplementary material.

$$\frac{L.M[x] = v}{\langle S, L, x\rangle \rightarrow_E \langle S, L, v\rangle} \text{ VAR-LOOKUP-RULE}$$

$$\frac{E_1 \rightarrow_E E_1'}{\langle S, L, E_1 + E_2\rangle \rightarrow_E \langle S, L, E_1' + E_2\rangle} \text{ AEXPR-ADD-RULE-1}$$

$$\frac{E_1 \in Val \wedge E_2 \rightarrow_E E_2'}{\langle S, L, E_1 + E_2\rangle \rightarrow_E \langle S, L, E_1 + E_2'\rangle} \text{ AEXPR-ADD-RULE-1}$$

$$\frac{E_1 \in Val \wedge v_1 + v_2 \rightarrow_E v_3}{\langle S, L, v_1 + v_2\rangle \rightarrow_E \langle S, L, v_3\rangle} \text{ AEXPR-ADD-RULE-2}$$

Fig. 9. Partial expression semantic rules for *Koord*.

3.4 Synchronization and consistency

Following our semantic rules in Section 3.3, careful readers would notice that all event transitions of *Koord* program take *zero* time. The environment transitions however take δ time for the evolution of the sensor and actuator ports together with the update of the local context from the global context.

To reiterate, the following are the timing requirements from rule EventTrans and EnvTrans: (a) an event transition takes *zero* time, (b) new values of controller ports are sampled at the end of each round (c) shared variables should reach consistent values within δ time, and (d) a global clock is used to synchronize each δ-time round. The first two requirements are achievable if the time taken to complete a program transition is negligible compared to δ, and δ can be a common multiple of the sampling intervals of all controller ports in use. These constraints are reasonable when computation and communication is comparatively much faster. Using the *Motion* module as an example, our position sensor on each device publishes every 0.01 sec (100Hz) while the CPU on each drone is 1.4 GHz. If we set δ to be 0.01 sec, an event transition taking 10K CPU cycles is still less than 0.1% of δ.

Requirement (c) and (d) are common research questions in distributed computing with an extensive literature. A global clock can be achieved with existing techniques that synchronize all local clocks on robots. The toolchain in [Author 2019]uses message passing to implement distributed shared memory for shared variables. It ensures that the time taken to propagate values through messages and reach consistency is smaller than δ, and the update is visible in the next round of program transitions for all robots. We therefore conclude our round based semantic with shared memory is a reasonable abstraction.

4 VERIFYING *KOORD* PROGRAMS

We have built the semantics of *Koord* in the $\mathbb{K}$ framework to enable decoupled analyses of platform-independent discrete part and the platform-dependent (dynamic) parts of distributed multi-robot systems. The *events* in an *Koord* program define the discrete computations in the system. The effect of a robot i executing event $e \in Events$ on a configuration $\boldsymbol{c} \in \mathbb{C}$, can be seen as a $\rightarrow_{stmt}$ application to $\langle \boldsymbol{c}.S, \boldsymbol{c}.L_i, Body\rangle$, where e is *"eventName:* **pre:** *Cond* **eff:** *Body"*.

4.1 Reachable configurations

Given a set of system configurations C, we define the following functions using the semantic rules of Section 3.3: (i) C_i^e returns the set of configurations obtained by robot i executing event $e \in Events$ from a configuration in C. (ii) C_i^+ returns the set of configurations obtained by robot i executing any event from a configuration in C. (iii) $C_{\bar{p}}^+$ returns all configurations visited, when robots execute

their events in the order $\vec{p}$, where $\vec{p}$ is a sequence of $p_i \in \mathtt{ID}$. (iv) C^+ is the union of $C^+_{\vec{p}}$ over all orders $\vec{p}$. (v) C^+_{prog} is the set of configurations reached from C *after* a program transition.

$$
\begin{aligned}
(C_i^+) &:= \{\boldsymbol{c}' \mid \exists \boldsymbol{c} \in C, Cond(\boldsymbol{c}.S, \boldsymbol{c}.L_i) \\
&\qquad \wedge \langle \boldsymbol{c}.S, \boldsymbol{c}.L_i, Body\rangle \rightarrow_{stmt} \left\langle \boldsymbol{c}'.S, \boldsymbol{c}'.L_i, \cdot \right\rangle\}, \\
C_i^{Skip} &:= \{\boldsymbol{c}' \mid \exists \boldsymbol{c} \in C, \langle \boldsymbol{c}.S, \boldsymbol{c}.L_i, \cdot\rangle \rightarrow_{stmt} \left\langle \boldsymbol{c}'.S, \boldsymbol{c}'.L_i, \cdot \right\rangle\} \\
C_i^+ &:= \bigcup_{\boldsymbol{c}\in C} \left(\boldsymbol{c}_i^{Skip} \cup \bigcup_{e \in Events} (\boldsymbol{c}_i^e) \right), \\
C_{\vec{p}}^+ &:= \begin{cases} \emptyset, & \text{if } \vec{p} = () \\ (C_{p_0}^+)_{\vec{p}'}^+, & \text{if } \vec{p} = (p_0, \vec{p}') \end{cases} \\
C^+ &:= \bigcup_{\vec{p} \in perms(\mathtt{ID})} C_{\vec{p}}^+, \\
C_{\mathsf{prog}}^+ &:= \left\{ \boldsymbol{c} \in (C^+) \mid \forall i \in \mathtt{ID}, \boldsymbol{c}.L_i.turn \neq \mathtt{prog} \right\}.
\end{aligned}
$$

All the above definitions can be restricted naturally to individual configurations. In the above, a sequence $\vec{p} = (p_0, \vec{p}')$, is written as a concatenation of the first element p_0 and the suffix $\vec{p}'$. Also, $perms(\mathtt{ID})$ refers to the set of permutations of $\mathtt{ID}$.

Next, we define the configurations that are reached during and after an environment transition. Recall that environment transitions capture the evolution of the actuator ports over a time interval $[0, \delta]$—all other parts of the configuration remain unchanged. Our *Koord* semantics defines the environment transitions with a *parameter* which is a (possibly black-box) function that captures the dynamics of individual robots.[2] Given such a function f_i for each robot i, we define the function $traj : \mathbb{C} \times [0, \delta] \mapsto \mathbb{C}$ to represent the evolution of the system over a $[0, \delta]$ time interval. $traj$ is constructed by simply updating all controller ports cp of each robot i using the function f_i that captures their respective dynamics. That is,

$$
\boldsymbol{c}' = traj(\boldsymbol{c}, t) \Leftrightarrow \left(\begin{array}{l} \forall i \in \mathtt{ID}, \boldsymbol{c}'.L_i.cp = f_i(\boldsymbol{c}.L_i.cp, t) \\ \wedge\ \boldsymbol{c}'.L_i.M = \boldsymbol{c}.L_i.M \\ \wedge\ \boldsymbol{c}'.L_i.turn = \boldsymbol{c}.L_i.turn \\ \wedge\ \boldsymbol{c}'.S = \boldsymbol{c}.S \wedge \boldsymbol{c}'.\tau = \boldsymbol{c}.\tau \\ \wedge\ \boldsymbol{c}'.turn = \boldsymbol{c}.turn \end{array} \right)
$$

Notice those additional constraints making sure all other fields of $\boldsymbol{c}$ and $\boldsymbol{c}'$ stay the same. We will use $\wedge \ldots$ to skip these kind of constraints in the later sections for simplicity. The set of system configurations $C(t)$ reached in an interval $[0, t]$ from C:

$$
C(t) := \{\boldsymbol{c}' \mid \exists \boldsymbol{c} \in C, 0 \leq t_1 \leq t, \boldsymbol{c}' = traj(\boldsymbol{c}, t_1)\}.
$$

We denote the set of points reached exactly the end of an environment transition from C is C_{env}.

Now to conform to our semantics, we carefully define the exact set of configurations reached right at the end of each *round* without transient configurations. A *frontier* set of configurations C^n represents those configurations that are reached from C *exactly when* n rounds are completed. Formally,

$$
C^n := \begin{cases} C, & \text{if } n = 0 \\ (C_{\mathsf{env}}^{n-1})_{\mathsf{prog}}^+ & \text{otherwise} \end{cases}
$$

[2]For different platforms, this function could be defined in closed form, as solutions of differential equations, or in terms of a numerical simulator.

Finally, given a set of configurations $C \subseteq \mathbb{C}$, the set of all reachable states in n rounds is defined inductively:

$$Reach(C, n) := \begin{cases} C, & if\, n = 0 \\ Reach(C, n-1) \cup \ (C^{n-1})^{+} \cup C^{n-1}_{\mathsf{prog}}(\delta), & \text{otherwise} \end{cases}$$

Notice that all transient configurations during both program and environment transitions are included in *Reach*.

4.2 Decomposing invariance verification

An *invariant* of a *Koord* program is a predicate that holds in all reachable configurations. Invariant requirements can express safety, for instance, that no two robots are ever too close (Collision avoidance), or that robots always stay within a designated area (Geofencing). Formally,

Definition 4.1. An invariant *inv* is a predicate (Boolean valued function) over a configuration or set of configurations such that, given a set of initial configurations of the system C_0,

$$\forall n \in \mathbb{N}, inv(Reach(C_0, n)),$$

where $inv(C)$ represents evaluating *inv* over each configuration in C.

Definition 4.2. A predicate *inv* is an *inductive invariant* of a system if given a set of initial configurations of the system C_0, the following proof obligations (POs) hold:

$$inv(C_0) \tag{Base}$$

$$inv(C) \Rightarrow inv(Reach(C, 1)) \tag{1}$$

That is, *inv* holds in the initial configuration(s) (PO (*Base*)), and *inv* is preserved by both platform-independent discrete program transitions and the platform-dependent environment transitions (PO (1)). It is straightforward to prove that an inductive invariant is an invariant.

Our verification strategy for user-specified (inductive) invariants is to discharge the proof obligations. PO (*Base*) is usually trivial. Therefore, we focus on PO (1). The *Koord* semantics enables us to *decouple* the environment and program transitions in *Reach*, and analyze each separately. PO (1) can be restated as

$$\forall \boldsymbol{c} \in \mathbb{C}, inv(\boldsymbol{c}) \Rightarrow inv(\boldsymbol{c}^{+}) \tag{2}$$

$$\forall \boldsymbol{c} \in \mathbb{C}, inv(\boldsymbol{c}) \Rightarrow \forall t \in [0, \delta], inv(\boldsymbol{c}^{+}(t)) \tag{3}$$

As in other concurrent systems, a major bottleneck in computing *Post* for PO (2) is the required enumeration of all $\vec{p} \in perms(\mathtt{ID})$ permutations for all robots with reads/writes to shared contexts. We therefore seek for a stronger and easier to prove proof obligation using the lemma below:

Lemma 4.3. *Given any inductive predicate φ, for any configuration c satisfying φ, the following always holds*

$$\left(\bigwedge_{i \in \mathtt{ID}} \bigwedge_{e \in Events} \forall \boldsymbol{c}' \in \boldsymbol{c}_i^e, \varphi(\boldsymbol{c}') \right) \Rightarrow \forall \boldsymbol{c}' \in \boldsymbol{c}^{+}), \varphi(\boldsymbol{c}')$$

The proof follows from expanding the definition of C^{+} and inducting on each event sequence. As φ is preserved before and after every event transition of every robot, the order of robot events do not violate φ. With Lemma 4.3, we strengthen and rewrite PO (2) as $\forall \boldsymbol{c}, \boldsymbol{c}' \in \mathbb{C}$,

$$\bigwedge_{i \in \mathtt{ID}} \bigwedge_{e \in Events} inv(\boldsymbol{c}) \wedge \boldsymbol{c}' \in \boldsymbol{c}_i^e \Rightarrow inv(\boldsymbol{c}') \tag{4}$$

which no longer requires enumeration of all permutations. This is the main reason how our synchronous model of execution helps verification scale.

We now discuss our approach to discharge PO (3). To further decouple program and environment transitions, we rewrite PO (3) as $\forall \boldsymbol{c}, \boldsymbol{c}', \boldsymbol{c}''$

$$(inv(\boldsymbol{c}) \wedge \ \boldsymbol{c}' \in \boldsymbol{c}^{+}_{\mathsf{prog}} \wedge \forall t \in [0, \delta], \boldsymbol{c}'' \in \boldsymbol{c}'(t) \Rightarrow inv(\boldsymbol{c}''). \tag{5}$$

PO (5) requires reasoning about the dynamic behavior of *traj* during environment transitions, and it is a challenging research problem by itself. We introduce *port assumptions* to abstract away the continuous dynamic behavior. Port assumptions allow users to over-approximate *traj* and prove the invariant at hand.

Actuator-Port assumptions discuss the restrictions on the behavior of sensor ports of the agents during the environment transitions given restrictions on sensor ports of the agent during the program transitions depending on how the actuator ports are set.

Definition 4.4. Given a configuration c', $c'.Acts$ refers to its actuator port values, and $c'.Sens$ refers to the sensor port values. Given the *traj* function an *Actuator-port assumption* $Act(P, Q)$ on a configuration c' is defined as

$$\forall t \in [0, \delta], P(c'.Acts, c'.Sens) \wedge c'' = traj(c', t) \Rightarrow Q(c''.Sens) \tag{AAsm}$$

where $A \subseteq Acts, S \subseteq Sens$.

where P is a boolean predicate defined over $CPorts \times Val \times CPorts \times Val$ and Q is a boolean predicate over $CPorts \times Val$.

We demonstrate later in this section how actuator and program port assumptions can be validated with our *Koord* simulator or with other specialized tools for continuous dynamics.

We know by definition $\boldsymbol{c}^{+}_{\mathsf{prog}} \subseteq Post(\boldsymbol{c})$, we can apply Lemma 4.3 in a similar way. Hence, with Lemma 4.3, we can merge PO (4) and PO (5), add program and actuator port assumptions, and strengthen our proof obligation as: $\forall \boldsymbol{c}$,

$$\bigwedge_{i \in \mathtt{ID}} \bigwedge_{e \in Events} inv(\boldsymbol{c}) \wedge \boldsymbol{c}' \in \boldsymbol{c}^{e}_{i} \wedge \forall t \in [0, \delta], \boldsymbol{c}'' \in \boldsymbol{c}'(t) \wedge Act(P, Q) \wedge Prog(M, N) \Rightarrow inv(\boldsymbol{c}'') \tag{Ind}$$

(4) need to rewrite this

(4) where we can use our $\mathbb{K}$ symbolic execution semantics to construct the symbolic post configuration to represent $\boldsymbol{c}^{e}_{i}$ for each event e. Notice that PO (*Ind*) allows us to reason in per event fashion as well as per robot fashion.

Dealing with loops and module functions. Many programs can include module related functions, and loops. Proving invariants over such loops is by itself a well studied and difficult research problem. To mitigate this problem, we instruct our symbolic execution to treat module related functions such as computing distance between two points, and paths generated by path planners as *uninterpreted functions*, and we introduce *function summary* for these uninterpreted functions similar to port assumptions.

Definition 4.5. A *function summary* $F(x, y)$ for an uninterpreted function $f(x)$ is a predicate which we can prove the following proof obligation:

$$\forall x, F(x, f(x)) \tag{FSum}$$

where x can be extended according to the arity of f. Verification and generation of good function summaries is extensively discussed and widely used in software verification. We believe writing a good function summary requires strong domain knowledge in particular robot devices as well as the problem to be solved.

5 CASE STUDY: LINEFORM

In this section, we revisit the LineForm program in Section 2. As mentioned in Section 4.2, the symbolic post configuration generated by $\mathbb{K}$ represents a set of *system configurations*. For variables in $\boldsymbol{c}$, their *primed copies*, and their double primed copies represent the variables in $\boldsymbol{c}^+$, and $\boldsymbol{c}^+(t)$ respectively. We reference the stakeholders as (5) shown in Figure 1 to illustrate how our approach facilitates defined roles for each.

(5) : Word that Katie suggested is increasingly popular in robotics/ai type problems. Should we use actors instead?

Programmer: specification of the geofencing requirement. The proposed invariant (provided by a requirements engineer) for the current configuration ($inv(\boldsymbol{c})$) is as follows

$$I_i(c) := M.pos_i \in rect(\mathrm{p}_{min}, \mathrm{p}_{max}) \wedge x[i] \in rect(\mathrm{p}_{min}, \mathrm{p}_{max})$$

This invariant $I_i(c)$ is defined on the variable values of the ith agent at the configuration c. $I_i(c^+)$ refers to the same invariant, where $M.pos_i$ and $x[i]$ are replaced by their primed copies, i.e their values for the ith agent for configurations in c^+, and similarly for $I_i(c^+(t)$.

This invariant indicates that the position of each agent is always within $rect(\mathrm{p}_{min}, \mathrm{p}_{max})$, and that each agent updates its shared variable value to one within $rect(\mathrm{p}_{min}, \mathrm{p}_{max})$ as well.

We first try to prove this invariant, without any assumptions, only from the constraints generated through the symbolic execution of the program.

Prover : symbolic post computation and induction hypothesis. For LineForm, our tool symbolically executes the event *TargetUpdate* (for robot i) and *automatically* generates the following post event configuration constraint $PostE_i$:

$$PostE_i := \left(\begin{array}{l} \neg(i = N_{sys} - 1 \vee i = 0) \\ \wedge\ M.tgt'_i = (x[i-1] + x[i+1])/2 \wedge x'[i] = M.pos_i \\ \wedge\ u_vars \wedge M.pos''_i := f(M.pos'_i, M.tgt_i, t) \wedge t \in [0, \delta] \end{array} \right)$$

where f encapsulates *traj* and is treated as an uninterpreted function over $\mathbb{R} \times \mathbb{R}$. Note that the function *rect* can both be precisely defined as well as left uninterpreted.

The rest of the formula includes a subformula *u_vars* that ensures the values of unmodified variables are unchanged such as $M.pos'_i = M.pos_i$ and $x'[j] = x[j]$ for $j \neq i$.

Because there is only one event, the induction proof obligation, the tool automatically generates the following proof obligation PO (*Ind*) for LineForm:

Proof Obligation 1. $\bigwedge_{i \in \mathrm{ID}} I_i(c) \wedge PostE_i \Rightarrow I_i(c^+(t))$

Our prover returns that the negation of PO (*Ind*) is satisfiable, meaning that our proposed invariant is not inductive. The satisfying assignment serves as a counter example. However, the automatically generated proof obligation for any application doesn't include any sensor or actuator assumptions. Specifically, it doesn't contain any restrictions on $M.pos''_i, M.tgt''_i$ w.r.t any of the variables in the symbolic post configuration after the event.

V&V engineer : port assumptions. We use the *Actuator-port assumption*, $Act(P_i, Q_i)$ to PO (*Ind*) constrain the trajectory , where

$$P_i := M.pos'_i \in rect(\mathrm{a}, \mathrm{b}) \wedge M.tgt'_i \in rect(\mathrm{a}, \mathrm{b})$$
$$Q_i := M.pos''_i \in rect(\mathrm{a}, \mathrm{b})$$

where a and b are constants in $\mathbb{R}^3$.

Given these definitions, the Actuator-port assumption becomes

Actuator-Port Assumption 1.

$$Act(P_i, Q_i) := \forall t \in [0, \delta], M.pos_i \in rect(\mathrm{a}, \mathrm{b}) \wedge M.tgt_i \in rect(\mathrm{a}, \mathrm{b}) \wedge c'' = traj(c', t) \Rightarrow M.pos_i'' \in rect(\mathrm{a}, \mathrm{b})$$

where c' is the configuration P_i is evaluated on and c'' is the configuration Q_i is evaluated on. Note that this $c'' = traj(c', t)$ is already captured in $PostE_i$. Having added this, our tool returns that the negation of the formula is unsatisfiable, meaning that this actuator assumption is enough to prove our invariant.

Table 1 summarizes the verification of Proof Obligation 1 on systems with different N_{sys}.

N_{sys}	**dim**	T_K (s)	T_V (s)	**Safe**
3	1	4.90	9.09	✓
3	2	4.19	10.13	✓
4	1	4.79	12.21	✓
4	2	5.28	12.49	✓
4	3	5.06	12.77	✓
5	1	4.91	18.46	✓

N_{sys}	**dim**	T_K (s)	T_V (s)	**Safe**
5	2	5.60	18.91	✓
5	3	5.42	20.30	✓
10	1	10.92	32.34	✓
10	2	10.96	32.42	✓
10	3	11.34	33.61	✓
15	1	12.23	53.89	✓

Table 1. Summary of semantics based verification for LineForm. N is N_{sys}, T_K is the symbolic post computation time in $\mathbb{K}$, T_V is the time taken for construction of constraints and verification in Z3. Robots moving along a line are represented by **dim** = 1, along a plane by **dim** = 2, and in a 3D space by **dim** = 3.

V&V engineer: validating port assumptions. We now turn towards validating the assumptions themselves. DryVR [Fan et al. 2017], which uses numerical simulations to learn the sensitivity of the trajectories of the vehicle to changes in initial conditions, with a certain confidence level. Then it uses this sensitivity and additional simulations to either prove the given invariant (in our case the port assumption) or find a counter-example. Under certain robustness assumptions, this process is also guaranteed to terminate. We used *Koord* simulator to generate traces of a quadcopter moving from a set of (uncertain) initial conditions to a target waypoint. From these traces, DryVR computes the (θ, δ)-*reachtube* from a given set of *uncertain* initial configurations θ.

Definition 5.1. Given a set $\theta \in \mathbb{R}^n, t > 0$, a (θ, t)-reachtube is a sequence of time-stamped compact sets $\left\{(O_i, t_i)_{i=0}^k\right\}$ where $(\theta(t_i) \setminus \theta(t_{i-1})) \subseteq (O_i)$. Recall that given a set of initial configurations $\theta(t)$ refers to the set of configurations reachable from θ within time t.

We check the δ-reachtube against $rect(\mathrm{p}_{min}, \mathrm{p}_{max})$. Figure 10 shows unsafe port assumption verification scenarios for simulated quadcopters. With certain dynamics models, the quadcopter overshoots its target, and indeed, the required containment rectangle as seen in Figure 10 *(Left)* and Figure 10 *(Center)*. This violates the Actuator-port assumption 1, which essentially states that given that if the current position and the target of the quadcopter are within the rectangle, then the position of the quadcopter at any time between now and δ will be within the rectangle. A potential way to deal with the first two scenarios for the *programmer* would adjust the invariant, specifically the values of p_{min} and p_{max} to ensure that the actuator port assumption is met.

With a different dynamics model, the quadcopter satisfies the port assumption as seen in Figure 10 *(Right)*. We can use symmetry transformations and exploit the fact that the trajectories of the quadcopter are rotationally and translationally symmetric to extend this argument to any compact set of initial configurations.

In the next section, we demonstrate this verification approach on a distributed task allocation application.

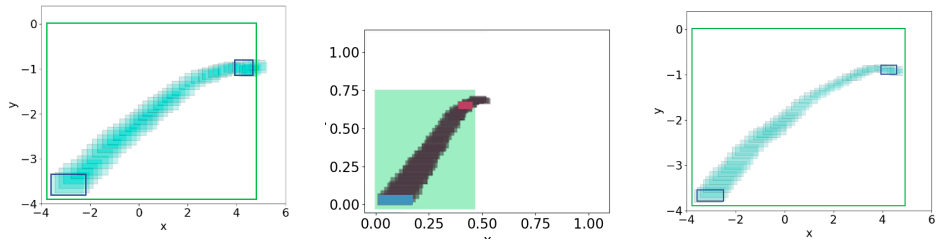

Fig. 10. All plots show the x vs y reachtube computations for the drone, with the green rectangle representing $rect(\mathrm{p}_{min}, \mathrm{p}_{max})$. The blue rectangle *Left* shows that the reachtube of the drone overshoots the rectangle. *Center* shows that the drone overshoots the rectangle, even with a different set of initial configurations. *Right* shows that with a different dynamics model, the actuator-port assumption is satisfied.

```
1  using Motion:
2    actuators:
3      pos[] path
4    sensors:
5      pos psn
6      boolean reached
7  local:
8    boolean isOnTask
9    Task myTask
10   pos[] tstRoute
11   int i
12
13 allread:
14   pos[] route
15 allwrite:
16   Task[] tasks
17
18 Assign:
19   pre: isOnTask == false
20   eff: if allAsgned(tasks):
21          stop
22     else: atomic:
23       for i = 0, length(tasks):
24         if task.assigned==false:
25           myTask=tasks[i]
26           tstRoute=
27              findPath(Motion.psn, myTask.loc)
28           if pathIsClear(route, tstRoute, pid):
29              isOnTask=true
30              assign(myTask, pid)
31              tasks[i]=myTask
32              route[pid]=tstRoute
33              Motion.path=tstRoute
34           else:
35              isOnTask=false
36              route[pid]=[Motion.psn]
37
38 Complete:
39   pre: isOnTask and Motion.reached
40   eff: isOnTask=false
41      route[pid]=[Motion.psn]
```

Fig. 11. *Koord* code for robot i for the Distributed Task Allocation

6 CASE STUDY : TASK

Task in Figure 11 is a simple *Koord* program to solve distributed task allocation problem. The problem statement is as follows: Given a set of (possibly heterogeneous) robots, a safety distance $\epsilon > 0$, and a fixed sequence of points (tasks) $list = x_1, x_2, \ldots \in \mathbb{R}^3$, there are following two requirements: (a) every unvisited x_i in the sequence is *visited* exactly by one robot and (b) no two robots ever get closer than ϵ. As in the previous case study, we discuss our approach through the defined roles for the stakeholders in Figure 1.

Programmer : specification of the safe separation requirement . In this case study, we only prove requirement (b) for Task. Task consists of two events (1) *Assign*, in which each robot looks for an unassigned task x from *list*; if there is a clear path to x then the robot assigns itself the task x, set the actuator port *Motion.path*, and shares its path with all other robots thru *shared_path*. Otherwise, it shares its position as the path. (2) *Complete*, which checks whether an robot has visited its assigned task. A path here is a list of points that a robot visits in sequence. The *Motion*

module drives the robot along a path, as directed by the position value set at its actuator port *Motion.path*. The sensor port *Motion.planner* returns a path to the target of an unassigned task, and a (user-defined) function called *pathIsClear* is used to determine whether the currently planned path is within ϵ distance of any path in *shared_path*.

Suppose there is a function parameterized by ϵ, taking two paths as input $clear_\epsilon : List\langle Point\rangle \times List\langle Point\rangle \mapsto bool$, it returns true only if the minimum distance between the two paths is greater than ϵ. We restate requirement (b) as:

Invariant 2. *No two robots ever get closer than ϵ. This gives us the following invariant for the current configuration c:*

$$I_i(c) = \forall j \in [N_{sys}], (i \neq j \wedge clear_\epsilon(shared_path[i], shared_path[j])) \vee (i = j)$$

Computing the *clear* function involves nested loops over the length of each path, then computing the minimum distance between each path segment *pathIsClear* further has to iterate over all shared paths and check via *clear*. We use the notion of *function summary* as defined in Section 4 to capture the notion of correctness for for *pathIsClear*. The function summary *PIC* is defined below as:

Function Summary 1.

$$PIC(sp, cp, i, y) := \forall j \in \texttt{ID}, j \neq i \wedge \neg clear_\rho(sp[j], cp) \Rightarrow \neg y$$

where $0 \leq \rho < \epsilon$. The function summary simply says, if my current path cp is not more than ϵ distance to any path $sp[j]$ shared by other robots, the output of $y = pathIsClear(sp, cp, i)$ should be false,[3] and PO (*FSum*) becomes:

Proof Obligation 2.

$$\forall sp, cp, i, PIC(sp, cp, i, pathIsClear(sp, cp, i))$$

Validating PO (*FSum*) requires reasoning about the implementation of the *pathIsClear* function, which is beyond the scope of this discussion. (6)

(6) phrase this differently

Prover : symbolic post computation and induction hypothesis. For constructing the symbolic set of configurations, we use a list with four tasks signified by $\{t_1, t_2, t_3, t_4\}$ so that the symbolic execution terminates. The for-loop iterating through the task list is unrolled into a sequence of (nested) *if-else* statements. For simplicity, we show the *automatically generated* symbolic post event configuration of the *Assign* event for only one execution when robot i picks t_1:

$$\begin{aligned} E_i^{t_1} := {} & \neg on_task_i \wedge on_task_i' \\ & \wedge curr_path_i' = M.planner(t_1.target) \\ & \wedge PIC(shared_path, curr_path_i', i, \texttt{True}) \\ & \wedge shared_path'[i] = curr_path_i{}' \\ & \wedge M.path_i' = shared_path'[i] \wedge u_vars \end{aligned}$$

where *u_vars* again, ensures the values of unmodified variables are unchanged. Notice how we can use *PIC* to summarize *pathIsClear*. Similarly, we get $E_i^{t_2}$, $E_i^{t_3}$ and $E_i^{t_4}$ for other execution paths choosing corresponding tasks. When none of the tasks is picked, the post event configuration generated is

$$E_i^{none} := \neg on_task_i \wedge shared_path'[i] = [M.pos_i] \wedge u_vars$$

[3]The index i in the *pathIsClear* function indicates that the previously shared path of the robot i shouldn't be considered in the computation.

Benchmark	N_{sys}	T_K (s)	T_V (s)	Safe
Task	3	9.90	10.6	✓
Task	4	9.79	11.78	✓
Task	5	9.91	14.92	✓
Task	10	12.92	18.34	✓

Table 2. Summary of semantics based verification for Task. T_K is the symbolic post computation time in $\mathbb{K}$, T_V is the time taken for generation of constraints and verification in Z3 and N_{sys} is the number of robots in the system.

For the event *Assign*, the post event configuration is:

$$Post_i := \left(\begin{array}{l} \forall j \in [N_{sys}], E_i^{t_1} \wedge E_i^{t_2} \wedge E_i^{t_3} \wedge E_i^{t_4} \wedge E_i^{none} \\ \wedge (M.pos'', M.reached'') = f(M.pos', M.reached', M.path', t) \wedge t \in [0, \delta] \end{array} \right)$$

Our prover then automatically generates the proof obligation :

Proof Obligation 3. $\bigwedge_{i \in \mathtt{ID}} I_i(c) \wedge Post_i \Rightarrow I_i(c^+(t))$

Without any assumptions on robot motion, our tool is unable to discharge this proof automatically.

V&V engineer : port assumptions. For abstracting the movement of robots, an robot should move closely ($\neg clear_\beta$, where $\beta + \rho \leq \epsilon$) along its *Motion.path* actuator whose value is denoted by $M.path$ until it finishes traversing the path. For simplicity, we denote *Motion.reached* by $M.reached$. We add $Act(P_i, Q_i)$ with

$$P_i := \neg M.reached_i'$$
$$Q_i := \neg clear_\beta(M.pos_i'', M.path_i'')$$

The corresponding Actuator-port assumption then becomes:

Actuator-Port Assumption 2.

$$Act(P_i, Q_i) := \forall t \in [0, \delta], \neg M.reached_i' \wedge c'' = traj(c', t) \Rightarrow \neg clear_\beta(M.pos_i'', M.path_i'')$$

The induction hypothesis for event *Complete* is generated similarly (omitted here), and the overall proof obligation is a conjunction of the two. Table 2 summarizes the verification of these constraints with different number of robots.

We now turn towards DryVR based validation for Actuator-port assumption 2. To do this, we computed reachtubes for our vehicle models and checked whether they were contained within β of the desired path. We found that the reachtube of the quadcopter satisfied this requirement, but our car model with did not, as seen in Figure 12(Right).

7 RELATED WORK

Early domain specific languages for robotics were proprietary and tied to specific platforms. See [Nordmann et al. 2014] for a detailed survey. With the lowering hardware costs and increasing popularity, there is a growing interest in open and portable frameworks and languages [Bohrer et al. 2018; Pinciroli et al. 2015; Williams et al. 2003; Zufferey 2017]. *Robot Operating System (ROS)* [Quigley et al. 2009] is the predominant member in this category. At its core, ROS supports a publish-subscribe-based communication and the ROS community has built drivers for numerous hardware components. Our implementation of the *Koord* abstractions for the quadcopter and vehicle platforms use ROS just like thousands of other robotics products and projects. The Table above gives a summary of robotics languages that have been deployed on hardware.

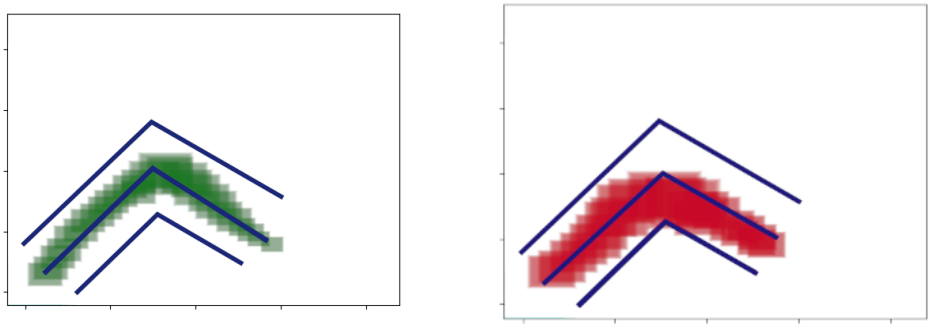

Fig. 12. *Left* The computed reachtube for the quadcopter lies within β of the actual path, thus the vehicle will not violate Actuator-port assumption 2.*Right* the computed reachtube for our car model is not contained so the vehicle may violate the assumption.

Framework /system	**Dist. Sys.**	**Hetero-geneous**	**Sim**	**Prog. Lang.**	**Compiler**	**V&V**
ROSBuzz [St-Onge et al. 2017]	✓	✓	✓	Buzz	✓	
PythonRobotics		✓	✓	Python		
PyRobot [Murali et al. 2019]		✓	✓	Python		
MRPT [Blanco 2009]		✓		C++		
Robotarium [Pickem et al. 2017]		✓	✓	Matlab		
Drona [Desai et al. 2017]	✓		✓	P [Desai et al. 2013]	✓	✓
Live [Campusano and Fabry 2017]		✓		LPR	✓	
Koord	✓	✓	✓	*Koord*	✓	✓

ROSBuzz [St-Onge et al. 2017] supports the Buzz language, which doesn't provide abstractions like *Koord* for path planning and shared variables. The Live Robot Programming language [Campusano and Fabry 2017] provides abstractions in terms of nested state machines and allows the program to be changed while running. It does not support robot ensembles. Programming systems using the shared memory paradigm have been developed for several distributed computing systems [Adve and Gharachorloo 1996; Calder et al. 2011; DeCandia et al. 2007; Lakshman and Malik 2010; Nitzberg and Lo 1991]. A position paper [Ghosh et al. 2018] proposed combining shared memory with physical interactions in a high-level language. This paper presents a full language, its formalization, and the proof system that combines those abstractions. P [Desai et al. 2013] and PSync [Drăgoi et al. 2016] are DSLs for asynchronous partially distributed systems, but cyber-physical interactions are not supported. P has been integrated into the DRONA framework [Desai et al. 2017] and the latter has very similar objectives to our work, but the approaches and solutions are different.

Drona is a framework for multi-robot motion planning and so far deployed only on drones. Koord and the underlying middleware it is deployed on aims to be more general, and multiple applications have been deployed on cars and drones in both simulations and hardware. The explicit model checker (using Zing) of Drona relies on manual proofs of their safe-plan-generator and path-executor, which are analogous to Koord function summaries and port assumptions. Drona's model checker explores reachable states upto a given depth (number of transitions from an initial state). Koord proves inductive invariants using our own symbolic executable semantics. Therefore, when all proof obligations are discharged for a candidate invariant, the Koord system proves the invariant holds for all reachable states. Further, while our Task application implements something

similar to the distributed plan generator which is a built-in feature for Drona, *Koord*'s port interfaces allow portability across arbitrary planners.

[Author 2019] presents the CyPhyHouse toolchain, including overview of the whole software stack on which Koord runs, with the focus on deploying Task application (2nd case study) on vehicle and quadcopter hardware. In contrast, this paper presents the formal description of *Koord* language and the key language and verification challenges.

8 CONCLUSIONS AND FUTURE WORK

Our goal in developing *Koord* was to provide a programming methodology for reliable distributed robotics that can exploit different analysis techniques while not requiring complete domain expertise. Our case studies on distributed formation control, task allocation, and mapping demonstrate how succinct *Koord* applications accomplish multi-robot coordination, sensing, actuation, and path planning. Moreover, the application programmer only needs to understand *Koord*'s shared memory semantics, and the sensor and actuator port abstractions. On the other hand, hardware engineers will need to validate that the port abstractions are indeed met by the target hardware platform through testing. We show through our case studies that symbolic execution of *Koord* programs helps in automating formal analysis of (distributed) platform independent programming logic, and verifiable platform dependent control properties can be plugged in as assumptions.

We showed in Section 4 how the semantics of *Koord* enables reduction of the concurrent analysis of inductive invariants preserved by event executions to sequential analyses, as event executions only perform reads from each agent's local memory. We also showed how simulation based sensitivity analysis (implemented in DryVR tool from the CPS community) can be used to validate/falsify port assumptions for a particular controller. Integrating such analyses into the *Koord* framework is left to interesting future work. out of place

Through our design of *Koord* semantics, case studies and experiments, we looked at the following research questions :

- *Can there be a separation of platform-dependent controller logic and independent programming logic in programming distributed robotics*
  In designing the formal semantics of *Koord*, and supporting the implementations of the platform specific abstractions that update sensor values, we found that we were able to "plug-and-play" several different dynamics models including quadcopters and ground vehicles with different sensing capabilities. The particular case studies we performed lent themselves well to separate formal analyses of the discrete and dynamic behaviors of the applications, and we were able to demonstrate the main components of an end-to-end verification approach using our prover for the discrete behaviors, and DryVR for simulation driven verification of of the dynamic behaviors.
- *How does the distributed nature of robotics applications affect their formal semantics and analysis?* Distributed robotics applications may have nondeterministic behaviors. The round based partially-synchronous design of *Koord* execution semantics ensures memory consistency every δ time units. Verification of distributed systems in general is a hard problem due to state space explosion, and usually is not achieved at the level of implementations in a programming language, and indeed, that is not the problem we solve. We found that inductive invariants, which were preserved during program transitions across *every event execution by any agent*, can be completely verified by our approach.
- *What degree of automation can this verification approach achieve ?*
  The symbolic post configurations can be computed automatically by our executable semantics, as well as the initial proof obligation when the programmer provides a candidate invariant.

Inductive invariants we presented in the case studies required port-assumptions, which we were able to come up with by observing the prover output due to our knowledge of the protocols. Discharging the proofs for those assumptions required domain specific knowledge of verification techniques for cyber-physical systems. Through this process, we were able to identify user-specific roles as well. In our case studies, we identified precisely which components of the verification were automated by our tool, and which were left to the user-specific roles.

It is difficult to expect that any language, including port assumption's, can cover vastly different types of robots (which are constantly evolving). To that end, our design on top of $\mathbb{K}$ semantics framework gives a flexible way to to extend our language and tailor it to specific robot types on demand. At the same time, it automatically adapts to the same framework for formal analysis. We plan to investigate this further on actual robots with diverse sensing and actuation capabilities. (7) We also plan to extend our work to include specification and verification of progress properties under fairness constraints for *Koord* applications.

(7) Dow we want to say that we plan to investigate this on concrete robots/-cars in the future?RG: Yes, should we add actual examples? robots with cameras, lidar, radar for pedestrian detection/interaction, robots with arms for warehouse apps

REFERENCES

Sarita V. Adve and Kourosh Gharachorloo. 1996. Shared memory consistency models: A tutorial. *IEEE Computer* 29 (1996), 66–76.

Hagit Attiya and Jennifer Welch. 2004. *Distributed Computing: Fundamentals, Simulations and Advanced Topics.* John Wiley & Sons, Inc., USA.

Anonymous Author. 2019. Anonymized Paper. In *to appear in ICRA 2020.* IEEE, Paris,France, 6 pages.

José-Luis Blanco. 2009. *Contributions to Localization, Mapping and Navigation in Mobile Robotics.* Ph.D. Dissertation. PhD. in Electrical Engineering, University of Malaga. http://www.mrpt.org/Paper:J.L._Blanco_Phd_Thesis

Timo Blender, Thiemo Buchner, Benjamin Fernandez, Benno Pichlmaier, and Christian Schlegel. 2016. Managing a Mobile Agricultural Robot Swarm for a seeding task. In *IECON 2016-42nd Annual Conference of the IEEE Industrial Electronics Society.* IEEE, 6879–6886.

Brandon Bohrer, Yong Kiam Tan, Stefan Mitsch, Magnus O. Myreen, and André Platzer. 2018. VeriPhy: Verified Controller Executables from Verified Cyber-physical System Models. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2018).* ACM, New York, NY, USA, 617–630. DOI:http://dx.doi.org/10.1145/3192366.3192406

Brad Calder, Ju Wang, Aaron Ogus, Niranjan Nilakantan, Arild Skjolsvold, Sam McKelvie, Yikang Xu, Shashwat Srivastav, Jiesheng Wu, Huseyin Simitci, Jaidev Haridas, Chakravarthy Uddaraju, Hemal Khatri, Andrew Edwards, Vaman Bedekar, Shane Mainali, Rafay Abbasi, Arpit Agarwal, Mian Fahim ul Haq, Muhammad Ikram ul Haq, Deepali Bhardwaj, Sowmya Dayanand, Anitha Adusumilli, Marvin McNett, Sriram Sankaran, Kavitha Manivannan, and Leonidas Rigas. 2011. Windows Azure Storage: A Highly Available Cloud Storage Service with Strong Consistency. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles (SOSP '11).* ACM, New York, NY, USA, 143–157. DOI:http://dx.doi.org/10.1145/2043556.2043571

Miguel Campusano and Johan Fabry. 2017. Live Robot Programming: The Language, its Implementation, and Robot API Independence. *Science of Computer Programming* 133 (2017), 1 – 19.

Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. 2007. Dynamo: Amazon's Highly Available Key-value Store. In *Proceedings of Twenty-first ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07).* ACM, New York, NY, USA, 205–220. DOI:http://dx.doi.org/10.1145/1294261.1294281

Ankush Desai, Vivek Gupta, Ethan Jackson, Shaz Qadeer, Sriram Rajamani, and Damien Zufferey. 2013. P: Safe Asynchronous Event-driven Programming. *SIGPLAN Not.* 48, 6 (June 2013), 321–332. DOI:http://dx.doi.org/10.1145/2499370.2462184

Ankush Desai, Indranil Saha, Jianqiao Yang, Shaz Qadeer, and Sanjit A Seshia. 2017. Drona: A framework for safe distributed mobile robotics. In *2017 ACM/IEEE 8th International Conference on Cyber-Physical Systems (ICCPS).* IEEE, 239–248.

Cezara Drăgoi, Thomas A. Henzinger, and Damien Zufferey. 2016. PSync: A Partially Synchronous Language for Fault-tolerant Distributed Algorithms. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '16).* ACM, New York, NY, USA, 400–415. DOI:http://dx.doi.org/10.1145/2837614.2837650

Chuchu Fan, Bolun Qi, Sayan Mitra, and Mahesh Viswanathan. 2017. DryVR: Data-driven verification and compositional reasoning for automotive systems. In *Computer Aided Verification (CAV).*

David Gauthier, Paul Freedman, Gregory Carayannis, and Alfred Malowany. 1987. Interprocess communication for distributed robotics. *IEEE Journal on Robotics and Automation* 3, 6 (1987), 493–504.

Mario Gerla, Eun-Kyu Lee, Giovanni Pau, and Uichin Lee. 2014. Internet of vehicles: From intelligent grid to autonomous cars and vehicular clouds. In *2014 IEEE world forum on internet of things (WF-IoT)*. IEEE, 241–246.

Ritwika Ghosh, Sasa Misailovic, and Sayan Mitra. 2018. Language Semantics Driven Design and Formal Analysis for Distributed Cyber-Physical Systems: [Extended Abstract]. In *Proceedings of the 2018 Workshop on Advanced Tools, Programming Languages, and PLatforms for Implementing and Evaluating Algorithms for Distributed Systems (ApPLIED '18)*. ACM, New York, NY, USA, 41–44. DOI:http://dx.doi.org/10.1145/3231104.3231958 event-place: Egham, United Kingdom.

Ge Guo and Wei Yue. 2012. Autonomous platoon control allowing range-limited sensors. *IEEE Transactions on vehicular technology* 61, 7 (2012), 2901–2912.

Avinash Lakshman and Prashant Malik. 2010. Cassandra: a decentralized structured storage system. *ACM SIGOPS Operating Systems Review* 44, 2 (2010), 35–40.

Steven M LaValle. 1998. Rapidly-exploring random trees: A new tool for path planning. (1998).

Nancy A. Lynch. 1996. *Distributed Algorithms*. Morgan Kaufmann Publishers Inc., Cambridge.

Pieter J Mosterman, David Escobar Sanabria, Enes Bilgin, Kun Zhang, and Justyna Zander. 2014. A heterogeneous fleet of vehicles for automated humanitarian missions. *Computing in Science & Engineering* 16, 3 (2014), 90.

Adithyavairavan Murali, Tao Chen, Kalyan Vasudev Alwala, Dhiraj Gandhi, Lerrel Pinto, Saurabh Gupta, and Abhinav Gupta. 2019. PyRobot: An Open-source Robotics Framework for Research and Benchmarking. *arXiv preprint arXiv:1906.08236* (2019).

B. Nitzberg and V. Lo. 1991. Distributed shared memory: a survey of issues and algorithms. *Computer* 24, 8 (aug. 1991), 52–60. DOI:http://dx.doi.org/10.1109/2.84877

Arne Nordmann, Nico Hochgeschwender, and Sebastian Wrede. 2014. *A Survey on Domain-Specific Languages in Robotics*. Springer International Publishing, Cham, 195–206. DOI:http://dx.doi.org/10.1007/978-3-319-11900-7_17

D. Pickem, P. Glotfelter, L. Wang, M. Mote, A. Ames, E. Feron, and M. Egerstedt. 2017. The Robotarium: A remotely accessible swarm robotics research testbed. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*. 1699–1706. DOI:http://dx.doi.org/10.1109/ICRA.2017.7989200

Carlo Pinciroli, Adam Lee-Brown, and Giovanni Beltrame. 2015. Buzz: An Extensible Programming Language for Self-Organizing Heterogeneous Robot Swarms. *CoRR* abs/1507.05946 (2015). http://arxiv.org/abs/1507.05946

J Norberto Pires and JMG Sá Da Costa. 2000. Object-oriented and distributed approach for programming robotic manufacturing cells. *Robotics and computer-integrated manufacturing* 16, 1 (2000), 29–42.

Morgan Quigley, Ken Conley, Brian P. Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Y. Ng. 2009. ROS: an open-source Robot Operating System. In *ICRA Workshop on Open Source Software*. IEEE.

Redmond R Shamshiri, Cornelia Weltzien, Ibrahim A Hameed, Ian J Yule, Tony E Grift, Siva K Balasundram, Lenka Pitonakova, Desa Ahmad, and Girish Chowdhary. 2018. Research and development in agricultural robotics: A perspective of digital farming. (2018).

Grigore Rosu and Traian Florin Serbanuta. 2014. K Overview and SIMPLE Case Study. In *Proceedings of International K Workshop (K'11) (ENTCS)*, Vol. 304. Elsevier, Illinois, 3–56. DOI:http://dx.doi.org/10.1016/j.entcs.2014.05.002

David St-Onge, Vivek Shankar Varadharajan, Guannan Li, Ivan Svogor, and Giovanni Beltrame. 2017. ROS and Buzz: consensus-based behaviors for heterogeneous teams. *CoRR* abs/1710.08843 (2017). http://arxiv.org/abs/1710.08843

Sebastian Thrun and others. 2002. Robotic mapping: A survey. *Exploring artificial intelligence in the new millennium* 1, 1-35 (2002), 1.

Brian C Williams, Michel D Ingham, Seung H Chung, and Paul H Elliott. 2003. Model-based programming of intelligent embedded systems and robotic space explorers. *Proc. IEEE* 91, 1 (2003), 212–237.

Damien Zufferey. 2017. The REACT language for robotics. (2017).